5OD21 – Optimisation Discrète
Projet : Course d'avions

Mohamed Benloughmari Fatimetou Narjah

1 Problème et graphe

On dispose de n aérodromes $V = \{1, \dots, n\}$, d'un départ d, d'une arrivée f, de K régions (on note V_k les aérodromes de la région k), d'un nombre minimal d'aérodromes A_{min} et d'un rayon d'action R. On cherche un chemin élémentaire de d à f, de longueur totale minimale, qui visite au moins A_{min} aérodromes (départ et arrivée compris), passe par chaque région et n'effectue que des sauts de longueur $d_{ij} \leq R$. Comme le suggère le sujet, nous partons de la modélisation du problème du voyageur du commerce orienté (§3.2.3 du cours). Deux différences : on ne visite pas tous les sommets, et l'on cherche un chemin $d \to f$ plutôt qu'un cycle hamiltonien.

On travaille sur le graphe orienté $G = (V, A)$ avec

$$A = \{(i,j) \in V^2 : i \neq j,\ d_{ij} \leq R,\ j \neq d,\ i \neq f\}.$$

La contrainte de rayon est ainsi prise en compte directement dans le graphe, et aucun arc n'entre en d ni ne sort de f. Pour $S \subseteq V$, on note $\delta^-(S) = \{(i,j) \in A : i \notin S,\ j \in S\}$ les arcs entrant dans S, $\delta^+(S)$ les arcs sortant de S, et $A(S) = \{(i,j) \in A : i \in S,\ j \in S\}$ les arcs internes à S (l'analogue orienté de $E(S)$ dans le cours). On écrit $\delta^{\pm}(k)$ pour $\delta^{\pm}(\{k\})$.

2 Formulations

Variables et contraintes communes. $x_{ij} \in \{0,1\}$ vaut 1 ssi l'avion va directement de i à j ; $y_i \in \{0,1\}$ vaut 1 ssi l'aérodrome i est visité.

$$\begin{aligned}
\min \quad & \sum_{(i,j)\in A} d_{ij}\, x_{ij} \\
\text{s.c.} \quad & \textstyle\sum_{\delta^+(d)} x_{dj} = 1, \qquad \sum_{\delta^-(f)} x_{if} = 1, \qquad y_d = y_f = 1 && (1)\\
& \textstyle\sum_{\delta^-(k)} x_{ik} = y_k, \qquad \sum_{\delta^+(k)} x_{kj} = y_k \qquad \forall k \in V \setminus \{d,f\} && (2)\\
& \textstyle\sum_{i\in V} y_i \geq A_{min} && (3)\\
& \textstyle\sum_{i\in V_k} y_i \geq 1 \qquad \forall k \in \{1,\dots,K\} && (4)
\end{aligned}$$

Les contraintes (2) sont celles du modèle d'affectation (§3.1.3), où le second membre 1 est remplacé par y_k. Elles traduisent aussi la relation logique « on ne peut utiliser un arc (i,j) que si i est visité » ($x_{ij} \leq y_i$, §3.1.2). La contrainte (4) est une contrainte de couverture (§3.1.4) : chaque région doit être couverte au moins une fois. Comme pour le voyageur du commerce, (1)–(2) n'empêchent pas les sous-tours : les arcs choisis forment un chemin $d \to f$ et, éventuellement, des circuits disjoints. Ces circuits permettent de « visiter » des aérodromes à bas coût pour satisfaire (3)–(4). Les deux formulations suivantes diffèrent uniquement par la manière de les interdire.

Formulation 1 : modèle ayant un nombre polynomial de contraintes (P_{pol}). Comme au §3.2.3, on ajoute une variable continue $u_i \in [1,n]$, le nombre de sommets visités depuis le départ d :

$$u_d = 1, \qquad u_j \geq u_i + 1 - n\,(1 - x_{ij}) \quad \forall (i,j) \in A. \tag{5}$$

Si $x_{ij} = 1$, on obtient $u_j \geq u_i + 1$; en sommant le long d'un circuit $i_1 \to \cdots \to i_p \to i_1$, on aurait $0 \geq p$, ce qui est impossible. Si $x_{ij} = 0$, n joue le rôle du majorant M (§3.1.2) et la contrainte n'est pas active. Ce modèle compte $|A| + 2n$ variables et $O(|A|)$ contraintes.

Formulation 2 : modèle d'élimination de sous-tours (nombre exponentiel de contraintes). Comme pour l'arbre couvrant et le voyageur du commerce (§3.2.1–3.2.2), on peut exprimer la connexité de deux façons. La *formulation coupe* (P_{coupe}) s'écrit

$$\sum_{(i,j)\in\delta^-(S)} x_{ij} \;\geq\; y_m \qquad \forall S \subseteq V \setminus \{d\},\ \forall m \in S \tag{6}$$

et la *formulation sous-tour* (P_{st}) s'écrit

$$\sum_{(i,j)\in A(S)} x_{ij} \;\leq\; \sum_{i\in S} y_i - y_m \qquad \forall S \subseteq V \setminus \{d\},\ \forall m \in S. \tag{7}$$

(6) dit que, si m est visité, au moins un arc entre dans tout ensemble S contenant m mais pas d : m est donc atteignable depuis d. Un sous-tour C viole (6) et (7) pour $S = C$. Avec $y_i = 1$ pour tout i, (7) redonne exactement la contrainte $\sum_{A(S)} x_{ij} \leq |S| - 1$ du cours. Le second membre devient $\sum_S y_i - y_m$ car les sommets ne sont pas tous visités.

Comparaison. Pour $S \subseteq V\setminus\{d\}$, tout sommet $j \in S$ vérifie $\sum_{\delta^-(j)} x_{ij} = y_j$ d'après (1)–(2). En sommant sur S, on obtient $\sum_{A(S)} x_{ij} + \sum_{\delta^-(S)} x_{ij} = \sum_{j\in S} y_j$, donc (6) $\Leftrightarrow$ (7), y compris pour (x, y) fractionnaire. Comme dans la propriété 3.2, les polyèdres associés aux relaxations continues sont égaux : $PV_{coupe} = PV_{st}$. Les deux formulations sont donc équivalentes (§3.7.3), et nous utilisons la forme coupe, plus simple à séparer. Le cours indique enfin que la formulation sous-tours est meilleure que la formulation polynomiale, au sens où elle fournit une borne de qualité au moins aussi bonne. Nous le vérifions numériquement.

Problème de séparation. Le nombre de contraintes (6) est de l'ordre de $n\,2^{n-1}$: on les ajoute uniquement lorsqu'elles sont violées. Étant donné (x^*, y^*) satisfaisant (1)–(4), il faut trouver $S \subseteq V \setminus \{d\}$ et $m \in S$ tels que $\sum_{\delta^-(S)} x^*_{ij} < y^*_m$, ou montrer qu'il n'en existe pas.

- *Cas fractionnaire.* On prend x^* comme capacités. Pour m fixé, $\min_{S\ni m,\ d\notin S} \sum_{\delta^-(S)} x^*_{ij}$ est la coupe minimale $d \to m$, égale au flot maximal de d à m (modèle de flot max du §3.6, théorème flot max / coupe min). Pour chaque m tel que $y^*_m > 0$, on calcule ce flot (Edmonds–Karp). S'il est strictement inférieur à y^*_m, l'ensemble S des sommets non atteignables depuis d dans le graphe résiduel donne une contrainte violée. La séparation est donc exacte et polynomiale (n calculs de flot).
- *Cas entier.* Les arcs choisis forment un chemin et des circuits. On suit le chemin depuis d ; chaque circuit C non atteint donne les contraintes violées $(S = C, m)$ pour tout $m \in C$.

3 Mise en œuvre

Les modèles sont écrits en Julia avec JuMP et résolus avec SCIP, avec une limite de 600 s par instance et par formulation. SCIP.jl ne prend pas en charge les *lazy callbacks* de JuMP (le code les utilise si l'on choisit Gurobi). Pour (P_{coupe}), nous résolvons donc une suite de PLNE : on résout le modèle avec les contraintes (6) déjà connues, on sépare les sous-tours de la solution entière, on ajoute les contraintes violées, puis on recommence. Chaque PLNE intermédiaire est une relaxation de (P_{coupe}), obtenue en optimisant sur un ensemble plus grand (§3.7.1). Dès que sa solution optimale ne contient plus de sous-tour, elle est réalisable pour (P_{coupe}), donc optimale.

La borne de relaxation continue à la racine z_{LP} est une borne inférieure (duale) de l'optimum. Pour (P_{pol}), on résout directement le PL relâché. Pour (P_{coupe}), on applique un algorithme de plans coupants sur le PL relâché, avec la séparation fractionnaire exacte, jusqu'à ce qu'aucune contrainte ne soit violée. Les contraintes trouvées sont conservées dans le premier PLNE, et le temps de cette phase est compté dans la limite de 600 s. Chaque solution est vérifiée indépendamment. Sur l'exemple du sujet (6 aérodromes), on retrouve l'optimum 12 avec le parcours $1 \to 3 \to 2 \to 5$. L'instance 90_1 déclare $K = 6$ mais attribue la région 7 à 5 aérodromes : nous respectons K et considérons ces aérodromes comme hors région.

4 Résultats

Le tableau 1 donne, pour chaque instance, la meilleure valeur connue z^* (optimale sauf mention), la borne de relaxation continue à la racine z_{LP}, l'écart $(z^* - z_{LP})/z^*$, le temps total et le nombre de nœuds du Branch-and-Cut. Pour (P_{coupe}), le nombre de nœuds est cumulé sur les PLNE successifs.

Instance	$\|A\|$	z^*	(P_{pol}) polynomiale z_{LP}	écart %	temps (s)	nœuds	(P_{coupe}) exponentielle z_{LP}	écart %	temps (s)	nœuds
0	18	12	8.0	33.3	0.0	1	10.0	16.7	0.3	1
20_1	288	73	65.2	10.8	0.0	1	71.0	2.7	0.1	1
20_2	286	91	63.0	30.8	0.7	67	74.5	18.1	0.0	1
20_3	113	82	64.0	22.0	0.0	1	72.5	11.6	0.0	1
20_4	226	96	78.0	18.8	0.1	10	86.5	9.9	0.0	1
30_1	156	115	84.1	26.9	0.6	623	110.5	3.9	0.9	2
40_1	417	124	114.2	7.9	0.0	1	124.0	0.0	0.6	1
50_1	441	148	137.0	7.4	0.3	1	148.0	0.0	1.6	1
70_1	757	288	231.1	19.8	7.7	1754	278.8	3.2	10.3	1
80_1	608	385	221.3	42.5	>600	491822	385.0	0.0	27.4	1
90_1	1287	605	409.8	32.3	>600	258278	604.3	0.1	62.0	2
100_1	957	665	429.5	35.4	>600	186610	658.7	0.9	54.4	2
100_2	1205	580	410.9	29.1	>600	208444	577.9	0.4	61.0	2
120_1	5473	481†	248.6	48.3	>600	67337	410.0	14.8	>600	0
150_1	1098	448	352.2	21.4	>600	243733	446.9	0.2	294.2	1

TABLE 1 – Résultats avec SCIP (limite 600 s). † meilleure solution trouvée, optimalité non prouvée : on a seulement l'encadrement $410.0 \leq z^* \leq 481$.

Lorsque (P_{pol}) atteint la limite de temps, on dispose seulement d'un encadrement entre la borne inférieure (duale) finale et la borne supérieure (primale) donnée par la meilleure solution admissible (§3.7.1). Pour (80_1, 90_1, 100_1, 100_2, 120_1, 150_1), on obtient respectivement : [367.8; 385], [545.0; 689], [639.1; 690], [559.3; 611], [388.7; 481] et [437.1; 456].

5 Analyse

Qualité des formulations. Sur toutes les instances, la borne de relaxation continue de (P_{coupe}) est au moins aussi bonne que celle de (P_{pol}), ce qui confirme que la formulation d'élimination de sous-tours est la meilleure des deux. L'écart moyen passe d'environ 26 % à environ 5.5 %. Il est inférieur à 1 % sur toutes les instances résolues avec $n \geq 80$, et nul sur 40_1, 50_1 et 80_1, où la solution optimale de la relaxation continue est déjà entière, donc optimale (§3.7.2). L'écart de (P_{pol}), au contraire, ne diminue pas quand n augmente et dépasse 30 % sur la plupart des grandes instances. On l'explique par le majorant $M = n$: avec x_{ij} fractionnaire, la contrainte $u_j \geq u_i + 1 - n(1 - x_{ij})$ n'est quasiment jamais active, et le polyèdre de la relaxation continue contient de nombreux points fractionnaires avec des sous-tours. Les contraintes (6) imposent au contraire, même en fractionnaire, qu'un flot y_m puisse être envoyé de d vers chaque sommet m.

Effort de résolution. Pour $n \leq 70$, les deux formulations résolvent les instances en quelques secondes. (P_{pol}) peut toutefois demander beaucoup de nœuds (1754 pour 70_1), alors que (P_{coupe}) termine presque toujours à la racine (1 ou 2 nœuds). À partir de $n = 80$, (P_{pol}) ne ferme plus l'encadrement en 600 s malgré plusieurs centaines de milliers de nœuds, et sa borne supérieure peut rester loin de l'optimum : 689 contre 605 pour 90_1, soit 14 % de plus. (P_{coupe}) résout ces instances en 30 s à 5 min. Le temps est presque entièrement consacré aux plans coupants de la racine : c'est le prix d'une bonne relaxation.

Limites. Le coût de la racine croît avec le nombre d'arcs. Sur 120_1 ($R = 90$, donc 5473 arcs, graphe presque complet), les plans coupants épuisent à eux seuls la limite de temps, et aucune solution admissible n'est obtenue avec (P_{coupe}). Sa borne inférieure (410.0) reste néanmoins meilleure que celle de (P_{pol}) après 600 s (388.7). Deux améliorations simples seraient de limiter le temps passé à la racine, ou d'utiliser un solveur qui accepte les *lazy callbacks* (Gurobi) pour séparer les contraintes pendant le Branch-and-Cut (option prévue dans le code).

Conclusion. Le modèle polynomial est simple, mais sa relaxation continue faible l'empêche de résoudre les instances au-delà de 70 aérodromes. Le modèle d'élimination de sous-tours, avec une séparation exacte par flot maximal, est une bien meilleure formulation : il résout 14 instances sur 15 à l'optimalité, contre 9.